

Contents

Full Stack Web Dev — Answer Key (Important Questions as per Mam)	1
PART A — 2 Mark Answers	1
Q1. Create a “FestivalGreeting” component that displays a greeting message . . .	1
Q2. Apply JSX to create a component that displays a welcome message with your name	2
Q3. React button that shows an alert “Registration completed!” on click	2
Q4. Button that toggles visibility of a message	2
Q5. Inline styling for a <div> with light-blue background and 10px padding . . .	3
Q6. React Router syntax for Dashboard and Contact Us pages	3
Q7. Student component: props (name, marks) → “Passed”/“Failed”	3
Q8. Controlled input for email using state	3
Q9. Differentiate controlled and uncontrolled components	4
Q10. HTTP status codes in the context of API responses in React	4
Q11. Using React DevTools to inspect a component’s state and props	5
Q12. React example handling errors while fetching student info (async API) — Scenario	5
Q14. Advantages of Axios over Fetch (error handling) — Scenario	5
Q15. React Router setup: home page & about us page — Scenario	6
Q16. useEffect fetching users from jsonplaceholder and saving to state — Scenario	6
PART B — Long Answers	7
Q1 (16 Marks) — Book Tracker Component	7
Q2 (16 Marks) — Employee List with map() and keys	7
Q3 (8 Marks) — Parent-Child Props + One-Way Data Flow	8
Q4 (8 Marks) — Star Rating + Virtual DOM Explanation	9
Q8 (8 Marks) — Background Color Toggle: Inline vs External CSS	10
Q9 (8 Marks) — Login Form: Controlled Components vs Traditional HTML Forms . . .	10
Q11 (8 Marks) — Sessions vs Cookies (Authentication & Preferences)	11
Quick Recap Table (What to prioritize revising)	12

Full Stack Web Dev — Answer Key (Important Questions as per Mam)

Based on your pattern: - **Part A (2 marks):** Set 1 → Q1-6, Set 2 → Q7-12 - **Part A Scenario (2 marks):** Set 1 → Q12 or Q14, Set 2 → Q15 or Q16 - **Part B (16 marks):** Q1 or Q2 - **Part B (8 marks):** Q3 or Q4, and (Set 2) Q8 or Q9 or Q11

PART A — 2 Mark Answers

Q1. Create a “FestivalGreeting” component that displays a greeting message

```
function FestivalGreeting() {  
  return <h2>Happy Diwali! Wishing you light and joy.</h2>;  
}  
export default FestivalGreeting;
```

Explanation: A functional component simply returns JSX. It needs no props/state here — it's a static display component.

Q2. Apply JSX to create a component that displays a welcome message with your name

```
function Welcome() {  
  const name = "Niranjan";  
  return <h1>Welcome, {name}!</h1>;  
}
```

Explanation: {name} embeds a JS variable inside JSX using curly braces — this is JSX expression interpolation.

Q3. React button that shows an alert “Registration completed!” on click

```
function RegisterButton() {  
  return (  
    <button onClick={() => alert("Registration completed!")}>  
      Register  
    </button>  
  );  
}
```

Explanation: onClick takes a function reference. An arrow function is used so alert() only runs on click, not immediately on render.

Q4. Button that toggles visibility of a message

```
import { useState } from "react";  
  
function ToggleMessage() {  
  const [visible, setVisible] = useState(true);  
  return (  
    <div>  
      <button onClick={() => setVisible(!visible)}>Toggle</button>  
      {visible && <p>Hello, I am visible!</p>}  
    </div>  
  );  
}
```

Explanation: visible && <p>...</p> is conditional rendering — the paragraph only renders when visible is true.

Q5. Inline styling for a <div> with light-blue background and 10px padding

```
<div style={{ backgroundColor: "lightblue", padding: "10px" }}>
  Styled Box
</div>
```

Explanation: In JSX, inline styles are a JS object ({ { } }), properties are camelCase, and values are strings.

Q6. React Router syntax for Dashboard and Contact Us pages

```
import { BrowserRouter, Routes, Route } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/dashboard" element={<Dashboard />} />
        <Route path="/contact" element={<ContactUs />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Explanation: BrowserRouter enables client-side routing; Routes/Route map a URL path to a component.

Q7. Student component: props (name, marks) → “Passed”/“Failed”

```
function Student({ name, marks }) {
  return (
    <p>{name}: {marks >= 35 ? "Student passed" : "Student failed"}</p>
  );
}
// Usage: <Student name="Ravi" marks={40} />
```

Explanation: Props are read via destructuring in the function parameter; the ternary operator picks the message conditionally.

Q8. Controlled input for email using state

```
import { useState } from "react";

function EmailInput() {
  const [email, setEmail] = useState("");
  return (
    <input
      type="email"
```

```

    value={email}
    onChange={(e) => setEmail(e.target.value)}
  />
);
}

```

Explanation: The input's value is bound to state, and onChange updates that state — this is what makes it a **controlled** component.

Q9. Differentiate controlled and uncontrolled components

Controlled	Uncontrolled
Value stored in React state	Value stored in the DOM itself
Updated via onChange + setState	Read using a ref when needed
Single source of truth = React	Single source of truth = DOM
More predictable, easier validation	Less code, closer to plain HTML

```

// Uncontrolled example
function Uncontrolled() {
  const inputRef = useRef();
  const handleSubmit = () => alert(inputRef.current.value);
  return <input ref={inputRef} />;
}

```

Q10. HTTP status codes in the context of API responses in React

Code range	Meaning	Example
2xx	Success	200 OK, 201 Created
3xx	Redirection	304 Not Modified
4xx	Client error	400 Bad Request, 401 Unauthorized, 404 Not Found
5xx	Server error	500 Internal Server Error

Explanation: When using fetch, only network failure triggers .catch() — a 404/500 still resolves successfully, so you must manually check response.ok or response.status and throw an error yourself:

```

fetch(url).then(res => {
  if (!res.ok) throw new Error(`Error: ${res.status}`);
  return res.json();
});

```

Q11. Using React DevTools to inspect a component's state and props

Steps: 1. Install the “React Developer Tools” browser extension. 2. Open browser DevTools → go to the **Components** tab. 3. Click any component in the tree — the right panel shows its current **props** and **state** live. 4. You can even edit a prop/state value directly in the panel to test how the UI reacts. 5. The **Profiler** tab additionally shows which components re-rendered and why (for performance debugging).

Q12. React example handling errors while fetching student info (async API) — Scenario

```
import { useState, useEffect } from "react";

function StudentInfo() {
  const [student, setStudent] = useState(null);
  const [error, setError] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    fetch("https://api.example.com/students/1")
      .then((res) => {
        if (!res.ok) throw new Error("Failed to fetch student data");
        return res.json();
      })
      .then((data) => setStudent(data))
      .catch((err) => setError(err.message))
      .finally(() => setLoading(false));
  }, []);

  if (loading) return <p>Loading...</p>;
  if (error) return <p>Error: {error}</p>;
  return <p>{student.name} — {student.marks} marks</p>;
}
```

Explanation: The fetch is wrapped so a non-OK response is turned into a thrown error, caught by `.catch()`, and shown via an error state — this prevents the app from crashing and gives the user feedback instead.

Q14. Advantages of Axios over Fetch (error handling) — Scenario

Aspect	Fetch	Axios
HTTP errors (404/500)	Does not reject the promise — you must check <code>res.ok</code> manually	Automatically rejects the promise on any non-2xx status, so <code>.catch()</code> works directly
JSON parsing	Manual: <code>res.json()</code>	Automatic: <code>response.data</code>
Timeout	Not built-in	Built-in via <code>timeout config</code>
Interceptors	Not built-in	Built-in (can globally handle auth errors, logging)

```
axios.get("/api/students")
  .then(res => console.log(res.data))
  .catch(err => console.log(err.response.status)); // works even for 404
```

Q15. React Router setup: home page & about us page — Scenario

```
import { BrowserRouter, Routes, Route, Link } from "react-router-dom";

function App() {
  return (
    <BrowserRouter>
      <nav>
        <Link to="/">Home</Link>
        <Link to="/about">About Us</Link>
      </nav>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<AboutUs />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Q16. useEffect fetching users from jsonplaceholder and saving to state — Scenario

```
import { useState, useEffect } from "react";

function Users() {
  const [users, setUsers] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/users")
      .then((res) => res.json())
      .then((data) => setUsers(data));
  }, []);

  return (
    <ul>
      {users.map((u) => <li key={u.id}>{u.name}</li>)}
    </ul>
  );
}
```

Explanation: The empty dependency array [] makes this run only once, when the component mounts — appropriate for a one-time data fetch.

PART B — Long Answers

Q1 (16 Marks) — Book Tracker Component

Requirement: Track borrowed book count starting at 0, with “Borrow Book” / “Return Book” buttons.

```
import { useState } from "react";

function BookTracker() {
  const [borrowedCount, setBorrowedCount] = useState(0);

  const handleBorrow = () => setBorrowedCount(borrowedCount + 1);
  const handleReturn = () => {
    if (borrowedCount > 0) setBorrowedCount(borrowedCount - 1);
  };

  return (
    <div style={{ textAlign: "center", padding: "20px" }}>
      <h2>Library Book Tracker</h2>
      <p>Books currently borrowed: <strong>{borrowedCount}</strong></p>
      <button onClick={handleBorrow}>Borrow Book</button>
      <button onClick={handleReturn} style={{ marginLeft: "10px" }}>
        Return Book
      </button>
    </div>
  );
}

export default BookTracker;
```

Explanation (for full marks): 1. **State management:** `useState(0)` creates a piece of state, `borrowedCount`, initialized to 0, plus its setter `setBorrowedCount`. 2. **Event handlers:** `handleBorrow` and `handleReturn` are separate functions (good practice over inline logic for readability) that call the setter with a new value. 3. **Guard condition:** Return Book checks `borrowedCount > 0` before decrementing, preventing negative counts — a small but important correctness detail. 4. **Re-rendering:** Every time `setBorrowedCount` is called, React schedules a re-render, and the updated `{borrowedCount}` value is reflected in the UI automatically — this is React’s **declarative UI** model: you describe *what* the UI should look like for a given state, not *how* to update the DOM manually. 5. **Immutability principle:** We never mutate `borrowedCount` directly (`borrowedCount++`) — we always call the setter with a new value, which is required for React to detect the change and re-render.

Q2 (16 Marks) — Employee List with `map()` and `keys`

Requirement: Component receives employee names as props, renders as a list using `map()`, explain key importance.

```
function EmployeeList({ employees }) {
  return (
    <ul>
      {employees.map((emp, index) => (
        <li key={emp.id ?? index}>{emp.name}</li>
      ))}
    </ul>
  );
}
```

```

    ))}
  </ul>
);
}

// Usage
function App() {
  const employees = [
    { id: 1, name: "Anita" },
    { id: 2, name: "Ravi" },
    { id: 3, name: "Kiran" },
  ];
  return <EmployeeList employees={employees} />;
}

```

Explanation (for full marks): 1. **map() for rendering lists:** `array.map()` transforms each data item into a JSX element (`<li>`), and returning an array of elements from inside `{}` tells React to render them all in sequence. 2. **What a key is:** `key` is a special prop React uses internally to identify *which* item in a list changed, was added, or was removed — it is not passed down to the component/element as a regular prop. 3. **Why keys matter for performance:** - React uses a **Virtual DOM** — a lightweight in-memory copy of the real DOM. - When state/props change, React builds a new Virtual DOM tree and **diffs** it against the previous one (the “reconciliation” process). - For lists, without unique keys, React can only compare items by their **position** (index) — so if an item is inserted at the top, React may wrongly think every item changed, and re-render/re-create the DOM nodes for all of them. - With a **stable, unique key** (like a database id), React can match old and new list items correctly even if their order changes, updating only what’s genuinely new/changed/removed — this avoids unnecessary DOM operations and preserves component state (e.g., input focus) correctly. 4. **Best practice:** Always use a stable unique ID from your data (e.g. `emp.id`) as the key — using the array index as a fallback is acceptable only when the list is static and never reordered/filtered.

Q3 (8 Marks) — Parent-Child Props + One-Way Data Flow

Requirement: Department parent + FacultyDetails & StudentDetails children, explain one-way data flow.

```

function FacultyDetails({ faculty }) {
  return <p>Faculty: {faculty}</p>;
}

function StudentDetails({ student }) {
  return <p>Student: {student}</p>;
}

function Department() {
  const facultyName = "Dr. Suresh Kumar";
  const studentName = "Priya R";

  return (
    <div>
      <h2>IT Department</h2>
      <FacultyDetails faculty={facultyName} />

```



```

        <StudentDetails student={studentName} />
      </div>
    );
  }
}

```

Explanation: - The Department component holds the data (facultyName, studentName) and passes it **downward** to its children as props (faculty={facultyName}, student={studentName}). - **One-way data flow** means data can only travel from parent → child, never automatically from child → parent. A child **cannot** modify the prop it receives directly. - If a child needs to send information back up (e.g., a form input's value), the parent must pass down a **callback function** as a prop, which the child calls — the data still technically flows “down” (the function itself is a prop), keeping the flow predictable. - **Why this matters:** it makes the app's data easy to trace and debug — you always know a value originated from some ancestor component, rather than being changed unpredictably from anywhere in the tree.

Q4 (8 Marks) — Star Rating + Virtual DOM Explanation

```

import { useState } from "react";

function StarRating() {
  const [rating, setRating] = useState(0);

  return (
    <div>
      {[1, 2, 3, 4, 5].map((star) => (
        <span
          key={star}
          onClick={() => setRating(star)}
          style={{
            cursor: "pointer",
            color: star <= rating ? "gold" : "gray",
            fontSize: "28px",
          }}
        >
          ★
        </span>
      ))}
      <p>Your rating: {rating} / 5</p>
    </div>
  );
}

```

Explanation — Virtual DOM process: 1. Clicking a star calls setRating(star), updating state. 2. React does **not** immediately touch the real DOM. Instead, it re-runs the component function to build a new **Virtual DOM tree** (a JS object representation of the UI). 3. React **diffs** this new tree against the previous Virtual DOM tree (a process called **reconciliation**) to compute the minimal set of changes. 4. Only the specific elements whose color actually changed (stars now filled/unfilled) and the rating text are updated in the **real DOM** — the rest of the page is left untouched. 5. This selective-update approach is far faster than re-rendering the entire page, because real DOM operations are expensive, while Virtual DOM diffing happens in memory.

Q8 (8 Marks) — Background Color Toggle: Inline vs External CSS

```
import { useState } from "react";
import "./App.css"; // external CSS file

function ColorBox() {
  const [isRed, setIsRed] = useState(false);

  // Inline style approach
  const inlineStyle = {
    backgroundColor: isRed ? "red" : "white",
    padding: "20px",
  };

  return (
    <div>
      {/* Inline styling */}
      <div style={inlineStyle}>Inline Styled Box</div>

      {/* External CSS approach */}
      <div className={isRed ? "box red" : "box"}>External CSS Box</div>

      <button onClick={() => setIsRed(!isRed)}>Toggle Color</button>
    </div>
  );
}

/* App.css */
.box { padding: 20px; margin-top: 10px; }
.red { background-color: red; }
```

Explanation — Inline vs External CSS: | Aspect | Inline Style | External CSS | |—|—|—| |
Syntax | JS object, camelCase (backgroundColor) | Normal CSS file, kebab-case (background-color) |
Dynamic updates | Very easy — just change the object based on state | Requires toggling className conditionally |
Reusability | Not reusable across components | Reusable via class names |
Pseudo-classes/media queries | **Not supported** (:hover, @media can't be inline) | Fully supported |
Performance at scale | Can get messy/slow with many dynamic styles | Better organized, browser can cache/optimize |

Conclusion: Inline styles are convenient for **small, highly dynamic** style changes tied directly to state (like this color toggle), but external CSS (or CSS Modules) is better for maintainability, reusability, and anything needing hover/media-query behavior.

Q9 (8 Marks) — Login Form: Controlled Components vs Traditional HTML Forms

```
import { useState } from "react";

function LoginForm() {
```

```

const [username, setUsername] = useState("");
const [password, setPassword] = useState("");
const [error, setError] = useState("");

const handleSubmit = (e) => {
  e.preventDefault();
  if (!username || !password) {
    setError("Both fields are required");
    return;
  }
  setError("");
  console.log("Logging in:", username);
};

return (
  <form onSubmit={handleSubmit}>
    <input
      type="text"
      placeholder="Username"
      value={username}
      onChange={(e) => setUsername(e.target.value)}
    />
    <input
      type="password"
      placeholder="Password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
    />
    {error && <p style={{ color: "red" }}>{error}</p>}
    <button type="submit">Login</button>
  </form>
);
}

```

Explanation — React (controlled) vs traditional HTML forms: | Aspect | Traditional HTML form | React controlled form | |—|—|—| | Value storage | Lives in the DOM (browser manages input state) | Lives in React state (useState) | | Reading values | Requires DOM queries (document.getElementById(...).value) or full page submission | Directly available as JS variables (username, password) | | Validation | Often done on submit only, or needs manual JS wiring | Can validate **live**, on every keystroke, since you control the state | | Submission | Default behavior reloads the page | e.preventDefault() stops reload; React handles submission via JS (e.g., API call) | | Role of controlled components | N/A | Makes React the **single source of truth** for form data, enabling instant validation feedback, conditional rendering (e.g., disabling the submit button), and easy integration with API calls — all without touching the DOM directly. |

Q11 (8 Marks) — Sessions vs Cookies (Authentication & Preferences)

Concept overview (general web dev, not React-specific): - **Cookies:** Small key-value data stored **in the browser**, sent automatically with every HTTP request to the same domain. Commonly used for storing simple, non-sensitive preferences (theme, language) or a session identifier. - **Sessions:** Data stored **on the server**, with only a session ID (usually via a cookie) kept on the client. The server looks up the full session data using that ID on each request.

```
// Express.js example – setting a cookie for preferences
res.cookie("theme", "dark", { maxAge: 900000 });

// Express-session example – authentication session
app.use(session({ secret: "mySecret", resave: false, saveUninitialized: true }));
app.post("/login", (req, res) => {
  req.session.userId = user.id; // stored server-side
  res.send("Logged in");
});
```

Differences (for full marks): | Aspect | Cookies | Sessions | |—|—|—| | Storage location | Client (browser) | Server (client only holds session ID) | | Data size limit | ~4KB per cookie | Can store much larger/richer data | | Security | More exposed (can be read/tampered client-side unless httpOnly/secure flags used) | More secure — actual data never leaves the server | | Lifespan | Can persist across browser restarts (maxAge) | Typically expires when the browser closes or after server-defined timeout | | Typical use | Preferences (theme, language), tracking | Authentication state, cart data, sensitive user info |

Conclusion: Cookies are best for small, low-sensitivity client-side data (like a theme preference), while sessions are the right choice for anything security-sensitive, like login/authentication state, since the actual data stays server-side.

Quick Recap Table (What to prioritize revising)

Marks	Questions	Core skill tested
2	Part A: 1–12, 14, 15, 16	Small isolated JSX/hooks/router snippets
16	Part B: 1 or 2	State management + list rendering & keys
8	Part B: 3 or 4	Props/data flow + Virtual DOM reasoning
8	Part B: 8, 9, or 11	Styling/forms/session vs cookies comparisons

Good luck — these cover everything flagged as important.